MESA SCHOOL OF BUSINESS

Data Foundations · Pre-Read

Session 1: Rapido

Big Mastery Question: We can only afford to add captains in 3 of our 10 cities next month. Which 3, and how do you know?

Before you come to class

Read this once, slowly. Don't memorize the syntax — just get comfortable with what each idea DOES and why a startup would ever need it. We'll build all of this live, together, in class. This document is so class-time goes straight to practice instead of definitions. Every sample table below shows real column letters (A, B, C...) across the top and real row numbers (1, 2, 3...) down the side — exactly like your actual Google Sheet — so you can see precisely how each formula is written.

The company: Rapido is India's bike-taxi and auto-rickshaw app — you tap, a captain (driver) shows up, you get somewhere cheap and fast. Rapido operates in 10+ cities, and every single ride creates one row of data: who rode, who drove, how far, how long, what it cost. Your job this session: use that rides log to decide which 3 cities deserve more captains next month. Nine Google Sheets ideas will get you there. Let's meet them one at a time.

0. The Dataset You'll Be Working With

Every example below pulls from the same shape of data you'll open in class — Rapido's rides log. Here's a small slice of it. Notice the gray strip on top showing column letters (A–F) and the gray strip on the left showing row numbers (1–7) — that's exactly how it will look on your screen, and it's how every formula below is written.

	A	B	C	D	E	F
1	ride_id	city	captain_id	distance_km	fare	rating
2	R1001	Bengaluru	C204	4.2	112	5
3	R1002	Hyderabad	C118	7.8	168	4
4	R1003	Bengaluru	C204	2.1	68	
5	R1004	Pune	C331	11.5	241	5
6	R1005	Bengaluru	C097	3.4	95	3
7	R1006	Hyderabad	C118	6.0	134	5

A 6-row sample of the Rapido rides log — the real sheet in class will have thousands of rows. Column A = ride_id, B = city, C = captain_id, D = distance_km, E = fare, F = rating.

1. Cells, Rows, Columns & Ranges

Real world reference:

Imagine a huge egg carton. Each little pocket holds exactly one egg — that pocket is called a CELL. All the pockets going sideways in one line are a ROW (like one shelf). All the pockets going up-and-down are a COLUMN (like one

lane). And if you scoop up a bunch of pockets together — say, 5 pockets in a row — that whole scoop is called a RANGE.

In Rapido's world:

In the rides log, every ROW is one ride (one trip someone took). Every COLUMN is one piece of information about that ride — the city, the fare, the rating. So the cell where ride R1003's fare lives is one specific pocket in the egg carton. If you wanted "every fare charged across all six rides," that's a RANGE — a group of cells, not just one.

Syntax:

A single cell is named by its COLUMN LETTER followed by its ROW NUMBER — no space, no symbol in between.

```
E4        → the cell in column E, row 4 A range is written as: B2:B7     → every cell from B2 down to B7
```

See it in the table:

Look back at the dataset grid in Section 0 above. Column E is "fare," and row 2 is ride R1001 — so cell E2 = 112. The range B2:B7 is every city name from row 2 to row 7, all six of them together. Here's the same grid again with a few specific cells called out:

	A	B	E
1	**ride_id**	**city**	**fare**
2	R1001	Bengaluru	112
3	R1002	Hyderabad	168
4	R1003	Bengaluru	68

Cell A2 = R1001. Cell E2 = 112 (the fare for R1001). The range B2:B4 = Bengaluru, Hyderabad, Bengaluru — every city shown here. Notice column B and column E are labelled with their REAL letters from the full sheet, even though column C and D aren't shown in this slice.

2. Relative vs Absolute References ($)

Real world reference:

Say you write a note on a sticky pad and stick it next to a toy. If you move the toy, the sticky note moves WITH it automatically — that's a RELATIVE reference, it shifts along with wherever you put it. Now imagine instead you nail a signpost into the ground that always points to the front door of your house, no matter where you're standing. That's an ABSOLUTE reference — it never moves, no matter what you drag around it.

In Rapido's world:

Say Rapido charges a flat 20% platform commission on every ride. You put that 20% (0.2) in one fixed cell — cell H1 — and write a formula that multiplies each ride's fare by that commission rate. If you copy that formula down 1,000 rows, you want every row to keep pointing back to that SAME H1 cell — not H2, not H3. That's when you "lock" the reference with a $ sign, so it stays put no matter how far down you copy it.

Syntax:

```
A1     → relative (shifts when copied) $A$1    → fully absolute (locked, never shifts) $A1     → column locked, row moves A$1     → row locked, column moves
```

See it in the table:

Cell H1 holds the commission rate, off to the side of the main data:

Cell H1 = 0.2 (the commission rate — set once, used by every row)

Now a new column, G, calculates commission for each ride using =E2*H1. Notice the E reference changes every row (relative), but H1 never changes (absolute):

	A	E	G
1	ride_id	fare	commission
2	R1001	112	=E2*H1 → 22.4
3	R1002	168	=E3*H1 → 33.6
4	R1003	68	=E4*H1 → 13.6
5	R1004	241	=E5*H1 → 48.2
6	R1005	95	=E6*H1 → 19.0

E2, E3, E4... changes every row (relative — correct, each row needs its own fare). But H1 never changes — it's locked to the one commission cell, off in column H.

3. Number & Currency Formatting

Real world reference:

It's like getting dressed up for a photo. The number underneath doesn't change — 112 is still 112 — but you can dress it up so it's instantly readable: adding a ₹ symbol so people know it's money, adding commas so big numbers don't look like a wall of digits, or fixing it to show exactly 2 decimal points so every price lines up neatly.

In Rapido's world:

A raw fare column showing "112" doesn't tell a founder if that's rupees, dollars, or paise. Format it once as currency, and every fare instantly reads ₹112.00 — no confusion, no extra typing needed for each row. This matters most when your sheet gets shared with someone who has 5 seconds to glance at it, like the founder deciding on captain expansion.

How to do it (menu, not a formula):

```
Select column E (fare) → Format → Number → Currency (or click the ₹ icon on the toolbar)
```

See it in the table:

Before formatting — column E is just plain numbers:

	A	E
1	ride_id	fare
2	R1001	112
3	R1002	168
4	R1003	68

After formatting column E as currency — same numbers, dressed up:

	A	E

1	ride_id	fare
2	R1001	₹112.00
3	R1002	₹168.00
4	R1003	₹68.00

4. Freeze Rows

Real world reference:

Imagine reading a really long comic strip that scrolls forever. Wouldn't it help if the title of the comic stayed glued to the top of the page no matter how far down you scrolled? That's exactly what FREEZING a row does — it glues your header row (the labels like "city" and "fare") to the top, so it never disappears as you scroll.

In Rapido's world:

The real rides log has thousands of rows. Without freezing, once you scroll past row 40 you'll have no idea which column is "fare" and which is "rating" anymore — you'd have to keep scrolling back up to check. Freeze row 1, and the headers stay visible forever, no matter how far you scroll.

How to do it (menu, not a formula):

```
View → Freeze → 1 row
```

See it in the table:

Below, imagine you've scrolled far down the sheet — row 1 (the header, shaded green) is still pinned at the top even though the data rows shown now are 5,502 to 5,504, not 2 to 4:

	A	B	E
1	ride_id	city	fare
5502	R5501	Chennai	121
5503	R5502	Chennai	189
5504	R5503	Kolkata	88

Notice the row numbers on the left jump straight to 5,502 — but row 1 (the header) is still visible above it, because it's frozen. That's the entire point.

5. Sort

Real world reference:

Think of lining up your toy cars from the smallest to the biggest, or your friends from shortest to tallest. SORTING just means putting rows in order, based on one column you choose — smallest to largest, or largest to smallest, or A to Z.

In Rapido's world:

You want to quickly see which rides earned the most money. Sort the whole rides log by the "fare" column, largest to smallest, and instantly the highest-paying rides float to the top — no formula needed, just reordering the rows.

How to do it (menu, not a formula):

```
Select the data → Data → Sort range → Sort by column E (fare), Z→A
```

See it in the table:

Before sorting — rows 2 to 5 in their original order:

	A	B	E
1	ride_id	city	fare
2	R1001	Bengaluru	112
3	R1002	Hyderabad	168
4	R1003	Bengaluru	68
5	R1004	Pune	241

After sorting by column E (fare), largest to smallest — the DATA moved, but the row numbers on the sheet are still just 2, 3, 4, 5:

	A	B	E
1	ride_id	city	fare
2	R1004	Pune	241
3	R1002	Hyderabad	168
4	R1001	Bengaluru	112
5	R1003	Bengaluru	68

6. Filter Views

Real world reference:

Imagine wearing magic sunglasses that hide everything except toys that are red. Everything else in the room is still there — you just can't see it right now. That's a FILTER: it temporarily hides rows that don't match what you're looking for, without deleting anything.

In Rapido's world:

You want to study Bengaluru rides only, without the noise of the other 9 cities. Apply a filter on column B (city), tick only "Bengaluru," and every other city's rows temporarily vanish from view — the data isn't gone, it's just hidden until you switch the filter off.

How to do it (menu, not a formula):

```
Select the data → Data → Create a filter → click the filter icon on column B (city)
→ untick every city except Bengaluru
```

See it in the table:

Before filtering — rows 2 to 6, all cities visible:

	A	B	E
1	ride_id	city	fare
2	R1001	Bengaluru	112

3	R1002	Hyderabad	168
4	R1003	Bengaluru	68
5	R1004	Pune	241
6	R1005	Bengaluru	95

After filtering to only "Bengaluru" — notice the row numbers on the left SKIP straight from 2 to 4 to 6, because rows 3 and 5 are hidden, not deleted:

	A	B	E
1	**ride_id**	**city**	**fare**
2	R1001	Bengaluru	112
4	R1003	Bengaluru	68
6	R1005	Bengaluru	95

7. SUM

Real world reference:

If you have a pile of blocks on the table and another pile on the chair, SUM is just pushing all the piles together and counting how tall the one big pile is. It adds numbers up.

In Rapido's world:

The founder asks: "How much total money did Bengaluru bring in this month?" You don't add fares one by one with a calculator — you SUM the entire fare column (or a filtered slice of it) and Sheets adds every single number instantly.

Syntax:

```
=SUM(range)  Example: =SUM(E2:E6)
```

See it in the table:

	A	E
1	**ride_id**	**fare**
2	R1001	112
3	R1002	168
4	R1003	68
5	R1004	241
6	R1005	95
7	**Total →**	**=SUM(E2:E6) = 684**

The formula sits in row 7, still column E — same column as the data it's adding up.

8. AVERAGE

Real world reference:

If you and 4 friends have different numbers of candies, and you want to share fairly so everyone has the same amount, AVERAGE tells you what that fair amount would be — add everything up, then split it evenly by how many people there are.

In Rapido's world:

Instead of just the total fare, the founder wants to know: "What does a typical Bengaluru ride cost?" That's an AVERAGE — total fare divided by number of rides — and it tells you far more about a normal customer's experience than the total alone.

Syntax:

```
=AVERAGE(range)  Example: =AVERAGE(E2:E6)
```

See it in the table:

	A	E
1	**ride_id**	**fare**
2	R1001	112
3	R1002	168
4	R1003	68
5	R1004	241
6	R1005	95
7	**Average →**	**=AVERAGE(E2:E6) = 136.8**

9. COUNT

Real world reference:

COUNT is like counting only your NUMBER toys in a toy box that has a mix of number toys and letter toys — you skip anything that isn't a number and just count how many actual numbers you see.

In Rapido's world:

Not every ride gets a rating — some riders skip it. If you want to know how many rides actually HAVE a numeric rating filled in, COUNT looks at column F (rating) and only counts the cells that hold a number, ignoring the blanks.

Syntax:

```
=COUNT(range)  Example: =COUNT(F2:F6)
```

See it in the table:

	A	F
1	**ride_id**	**rating**
2	R1001	5
3	R1002	4
4	R1003	(blank)
5	R1004	5

6	R1005	3
7	**Count →**	**=COUNT(F2:F6) = 4**

R1003 in row 4, column F, has no rating — so COUNT skips it. The answer is 4, not 5.

10. COUNTA

Real world reference:

COUNTA is like counting every single toy in the box — number toys AND letter toys AND anything else — as long as the spot isn't completely empty. It counts anything that has SOMETHING in it, not just numbers.

In Rapido's world:

You want to know how many rides were logged in total. Column A (ride_id) always has text like "R1001" — never a plain number — so COUNT (numbers-only) would return zero. COUNTA counts any filled cell, text or number, so it correctly tells you how many rides exist.

Syntax:

```
=COUNTA(range)  Example: =COUNTA(A2:A6)
```

See it in the table:

	A
1	**ride_id**
2	R1001
3	R1002
4	R1003
5	R1004
6	R1005
7	**=COUNTA(A2:A6) = 5**

11. MIN

Real world reference:

If you line up a bunch of ants from smallest to biggest, MIN just points at the tiniest ant in the whole line — the smallest value in a group of numbers.

In Rapido's world:

You want to check: what's the shortest ride ever recorded in the data? Maybe it's a data-quality flag (a 0.1 km ride might be an error), or maybe it's genuinely the shortest hop someone took. MIN finds that smallest number instantly.

Syntax:

```
=MIN(range)  Example: =MIN(D2:D6)
```

See it in the table:

	A	D
1	ride_id	distance_km
2	R1001	4.2
3	R1002	7.8
4	R1003	2.1
5	R1004	11.5
6	R1005	3.4
7	**Shortest →**	**=MIN(D2:D6) = 2.1**

12. MAX

Real world reference:

Same ant line, opposite question — MAX points at the biggest, tallest ant in the group. It finds the largest value.

In Rapido's world:

You want to spot the single highest fare ever charged — maybe it's a long airport run, or maybe it's a pricing glitch worth investigating. MAX pulls out that top number instantly, without you scrolling through every row by eye.

Syntax:

```
=MAX(range)  Example: =MAX(E2:E6)
```

See it in the table:

	A	E
1	ride_id	fare
2	R1001	112
3	R1002	168
4	R1003	68
5	R1004	241
6	R1005	95
7	**Highest →**	**=MAX(E2:E6) = 241**

13. Autofill

Real world reference:

Imagine you're drawing dots to continue a pattern — 1, 2, 3... and someone says "just keep going the same way all the way down the page." Autofill is Sheets doing that for you: you write one formula once, then drag it down, and it repeats the same logic on every row below, automatically adjusting for each new row.

In Rapido's world:

You've written the commission formula (=E2*H1) for ride R1001 in row 2, column G. You do NOT want to retype that formula 4,000 times for 4,000 rides. Instead, you click the tiny square at the bottom-right corner of cell G2 and drag it all the way down — Sheets copies the formula down every row, automatically updating E2 to E3, E4, E5... while keeping H1 locked exactly as we saw in Concept 2.

How to do it:

```
Click cell G2 (the cell with your formula) → grab the small square at its bottom-right corner (the "fill handle") → drag down to the last row Shortcut: select the range → Ctrl + D (fills down)
```

See it in the table:

	A	E	G
1	**ride_id**	**fare**	**commission**
2	R1001	112	=E2*H1 → 22.4 (typed by hand)
3	R1002	168	=E3*H1 → 33.6 (autofilled)
4	R1003	68	=E4*H1 → 13.6 (autofilled)
5	R1004	241	=E5*H1 → 48.2 (autofilled)
6	R1005	95	=E6*H1 → 19.0 (autofilled)

Only the formula in G2 was typed by hand. Every row below it was Autofill dragging the same pattern down.

Coming to class

You now know what each idea does, why Rapido would need it, and exactly which cell each formula lives in — you don't need to have memorized anything further. In class, we'll open the real rides log together, build these live, and use them to answer the big question: which 3 cities get more captains next month?